

HAUZ Backend Interview Task

Backend-focused implementation exercise with basic frontend integration.

Registration	Email verification	Login and logout
--------------	--------------------	------------------

Task

Build a small web application with these features:

1. Registration with email verification
2. Login with email and password
3. Logout

The registration form should collect:

- first name;
- last name;
- email;
- password;
- Personal Account role: `consumer` or `realtor`;
- contact phone, if the user wants to provide one.

After registration is completed, the application must create the correct HAUZ records using the provided schema.

The frontend only needs simple, clear forms and a basic authenticated page. We are not evaluating visual design or complex components.

Use

- TypeScript
- TanStack Start
- Appwrite Auth, Functions, and database (TablesDB)
- The provided HAUZ schema included in this document

Read the Appwrite documentation and figure out how the frontend, Appwrite Auth, Functions, and database should work together.

Appwrite

Correct use of Appwrite is an important part of this task. Spend time understanding the platform and how its Auth, Functions, and database components should be used and configured to work together.

Schema

The provided schema is final and contains everything needed for this task. Do not add, remove, rename, or change any table, field, relationship, index, or allowed value.

The `consumer` and `realtor` Personal Role records should be treated as existing data.

Features to implement

Implement the following features:

Registration

- Create a new account with email and password.
- Send and complete email verification.
- Support both Consumer and Realtor registration.
- Create the required HAUZ records correctly.
- Do not show registration as complete if verification or HAUZ record creation has failed.
- Handle invalid input, repeated requests, and failures safely.
- Do not leave partial or duplicate HAUZ records.

Login

- Allow a registered user to log in with email and password.
- Keep the user logged in after a page refresh.
- If registration or email verification is incomplete, show the correct next step.
- Show clear errors when login fails.

Logout

- Log out the current user.
- Clear the signed-in state.
- Do not allow logged-out users to open the authenticated page.

Submission

You have 24-48 hours to complete the task.

Host the code in your personal GitHub account and send us the repository link. The repository may be public or private; if it is private, give us access.

A deployed application is optional. However, the solution must work if we connect it to our own Appwrite project. Do not hard-code project-specific connection values or secrets in the application code.

Include a short [README.md](#) with the environment variables and Appwrite settings we need to run the application, along with any important assumptions or limitations.

What we will review

- Whether the implementation is correct
- How well the provided schema is understood and used
- How correctly the Appwrite platform and its components - Auth, Functions, and database - are used and configured
- How Appwrite Auth and HAUZ data are handled
- Code quality and structure
- Basic frontend-to-backend integration

Tests are not required.

Only build the features listed above. Password recovery, phone verification, MFA, OAuth, account switching, and other HAUZ features are outside the scope of this task.

HAUZ Authentication Schema

Personal Accounts

The entity relationship diagram is supplied separately as a self-contained HTML file.

Scope

This is the part of the HAUZ authentication schema needed for Personal Account registration.

In the real HAUZ platform, a registered human may have a Personal Account, one or more Agency Member Accounts with agency roles such as Owner, Manager, or Agent, and an administrative Staff Account. This task covers only Personal Accounts. Agency Member Accounts, Staff Accounts, and account switching are outside the scope of this task. So you are only provided with the schema that is necessary for the features you should implement.

Use this schema as provided. Do not add, remove, rename, or change anything.

Appwrite adds fields such as `$id`, `$createdAt`, `$updatedAt`, `$sequence`, `$databaseId`, `$tableId`, and `$permissions`. They are not repeated below. Use a generated UUID for each HAUZ row `$id`.

Appwrite Auth user

An Appwrite Auth user represents the registered human: the person who owns the email address, password, email verification status, and login session. Appwrite creates and stores this user through Appwrite Auth.

This is not a HAUZ table, and you should not create a table for it. Appwrite Auth returns the user's `$id`; HAUZ stores that value in `account_access_grants.appwrite_user_id` to connect the registered human to their HAUZ Accounts.

The distinction is:

- The Appwrite Auth user identifies who logged in.
- A HAUZ Account is a separate account that the registered human can use inside HAUZ.

One Appwrite Auth user can be connected to several HAUZ Accounts.

HAUZ tables

accounts

The main record for a HAUZ Account. It is separate from the Appwrite Auth user and stores the account type, current status, and which Account Access Grant is currently active.

This task only requires creating an `individual` Account and its Personal Account data. Other HAUZ Account types are outside the scope of this task.

Field	Type	Constraint
<code>account_type</code>	enum	Required and immutable: <code>individual</code>
<code>status</code>	enum	Required, no default: <code>active</code> <code>blocked</code>
<code>terminated_at</code>	datetime	Nullable; immutable once set
<code>current_access_grant_id</code>	varchar(36)	Nullable; stores an <code>account_access_grants.\$id</code> in a regular string field, not an Appwrite relationship

Indexes:

- `ix_accounts_type_status` (`account_type`, `status`)
- `ix_accounts_terminated` (`terminated_at`)

account_access_grants

This table connects the registered human in Appwrite Auth to the HAUZ Account they are allowed to use. It is required for this task even though account switching is not required. Without this row, there is no connection between the logged-in Appwrite Auth user and the Personal Account created for them.

The wider purpose of this table is to let one registered human use several separate HAUZ Accounts without needing a separate login for each one. For example, the same human may have a Personal Account and an Agency Member Account. Those accounts would have separate grants with the same `appwrite_user_id` and different `account_id` values.

Each row also represents one period of access. Setting `revoked_at` ends that access while keeping the row as history. An Account can therefore have older grants, so `accounts.current_access_grant_id` identifies the grant that currently provides access to it.

Field	Type	Constraint
<code>appwrite_user_id</code>	<code>varchar(36)</code>	Required; stores the registered human's Appwrite Auth user <code>\$id</code>
<code>account_id</code>	<code>relationship -> accounts</code>	Required
<code>revoked_at</code>	<code>datetime</code>	Nullable; immutable once set

Indexes:

- `-ix_grants_user_access (appwrite_user_id, revoked_at)`
- `-ix_grants_account_history (account_id, revoked_at)`
- `-ix_grants_user_account (appwrite_user_id, account_id, revoked_at)`

personal_accounts

Stores the personal profile and Personal Role for an `individual` Account.

Field	Type	Constraint
<code>account_id</code>	<code>relationship -> accounts</code>	Required and unique
<code>role_id</code>	<code>relationship -> personal_roles</code>	Required
<code>first_name</code>	<code>varchar(100)</code>	Required
<code>last_name</code>	<code>varchar(100)</code>	Required
<code>contact_phone</code>	<code>varchar(20)</code>	Nullable; E.164 when present

Indexes:

- `-uq_personal_account (account_id)`
- `-ix_personal_role (role_id)`
- `-ix_personal_contact_phone (contact_phone)`

personal_roles

Lists the Personal Roles available to Personal Accounts.

Field	Type	Constraint
<code>value</code>	<code>enum or varchar</code>	Required, unique, and immutable: <code>consumer</code> <code>realtor</code>

Index:

- `-uq_personal_roles_value (value)`

The `consumer` and `realtor` rows use generated UUIDs and should be treated as existing data.